Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110

(An Autonomous Institution, Affiliated to Anna University, Chennai)

B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024

III Year CSE (V Semester) Academic Year 2026-27

UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

1.Scalable E-Commerce System

Design and develop a prototype of a scalable E-Commerce System supporting user registration and authentication, product browsing and search, shopping cart management, order processing, product sharing, and competitor price monitoring. The system should be designed to handle increasing numbers of users, products, and requests while providing efficient and reliable services.

The following system-design components should be incorporated wherever applicable: Rate Limiter, Consistent Hashing, Key-Value Store, Sharding, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the E-Commerce functionalities is not provided. You should analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified.

You should develop an overall system architecture, design and implement the selected components, and integrate them into a working E-Commerce prototype. They should demonstrate important user flows such as login, product search, adding products to a cart, placing an order, and sharing a product link. The competitor price-monitoring functionality may be implemented as a background process using simulated competitor websites.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average response time, throughput (requests/second), success/error rate, and resource usage. Component-specific metrics

should also be reported where applicable. For example, request rejection rate for the rate limiter, load distribution across servers for consistent hashing, lookup latency for the key-value store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and suggestion accuracy for autocomplete.

Students should evaluate the system under at least three different workload levels say, low, medium, and high request loads) and present the measured results using tables or graphs. Where relevant, students should compare the proposed design with a simple baseline implementation to demonstrate the benefit of the selected system-design approach. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of users/products, adding a product server, or making one server unavailable.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges and map the appropriate components to the E-Commerce functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, data stores, system components, and their interactions.
3. Working Prototype – Implement and integrate the selected components into the E-Commerce application and demonstrate the major user flows.
4. Performance Evaluation – Report measurable metrics including response time, throughput, success/error rate, and relevant component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.
5. Scalability & Failure Analysis – Demonstrate how the system behaves when the workload or system size increases and under at least one relevant failure scenario. Discuss the observed bottlenecks and trade-offs.
6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental

results, design trade-offs, and limitations, followed by an end-to-end demonstration.

Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)
B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024
III Year CSE (V Semester) Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

2.Scalable Hospital Management System

Design and develop a prototype of a scalable Hospital Management System supporting patient registration, doctor and appointment management, patient records, pharmacy services, billing, and emergency services. The system should be designed to handle increasing numbers of patients, doctors, appointments, and concurrent requests while providing efficient, reliable, and timely access to hospital services.

The following system-design components are available for incorporation: Rate Limiter, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the hospital functionalities is not provided. Students must analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified.

The system should support functionalities such as patient registration and authentication, doctor search, appointment booking, patient record access, prescription and pharmacy management, billing, and emergency-service requests. Students should develop an overall system architecture, design and implement the selected components, and integrate them into a working prototype. For example, doctor and medicine search should support efficient search and suggestions, while patient and appointment information should remain accessible as the number of users and records increases.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average and 95th-percentile response time,

throughput (requests/second), success/error rate, and resource usage. Component-specific metrics should also be reported where applicable—for example, request rejection rate for the rate limiter, load distribution across hospital servers for consistent hashing, lookup latency for the key-value store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and accuracy for autocomplete. For the web crawler, students may use simulated hospital-related external sources, such as insurance-provider or medical-information websites, rather than real patient data.

Students should evaluate the system under at least three workload levels (for example, low, medium, and high numbers of concurrent requests) and present the measured results using tables or graphs. Where appropriate, students should compare the proposed design with a simple baseline implementation. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of patients or appointments, adding a hospital server, or making one server unavailable. No real patient or personally identifiable medical data should be used; students should use synthetic or generated data.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges in the Hospital Management System and map the appropriate components to the relevant functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, data stores, system-design components, and their interactions.
3. Working Prototype – Implement and integrate the selected components and demonstrate major workflows such as patient registration, doctor search, appointment booking, record access, and billing.
4. Performance Evaluation – Report measurable metrics including response time, 95th-percentile latency, throughput, success/error rate, and relevant

component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.

5. Scalability & Failure Analysis – Demonstrate system behaviour when workload or system size increases and under at least one relevant failure scenario. Discuss observed bottlenecks, scalability, reliability, and design trade-offs.
6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental results, trade-offs, limitations, and ethical/data-privacy considerations, followed by an end-to-end demonstration.

Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)
B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024
III Year CSE (V Semester) Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

3. Scalable Library Management System

Design and develop a prototype of a scalable Library Management System supporting member registration and authentication, book search and browsing, book availability, borrowing and returning, reservation management, and fine/payment management. The system should be designed to handle increasing numbers of members, books, transactions, and concurrent requests while providing efficient and reliable access to library services.

The following system-design components are available for incorporation: Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the library functionalities is not provided. Students must analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified.

The system should support functionalities such as member registration and authentication, book search, author/category search, book availability checking, borrowing and returning books, reservations, and fine management. Students should develop an overall system architecture, design and implement the selected components, and integrate them into a working prototype. The book and member data should be capable of being partitioned across multiple database shards, while consistent hashing may be considered for distributing requests or data across servers. The web crawler may be used with simulated external book sources to obtain information such as book details, ratings, or availability.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average response time, throughput (requests/second), success/error rate, and resource usage. Component-specific metrics should also be reported where applicable—for example, request rejection rate for the rate limiter, data distribution across

shards, load distribution across servers for consistent hashing, lookup latency for the key-value store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and accuracy for autocomplete.

Students should evaluate the system under at least three workload levels (for example, low, medium, and high numbers of concurrent requests) and present the measured results using tables or graphs. Where appropriate, students should compare the proposed design with a simple baseline implementation. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of members/books, adding a database shard or library server, or making one server unavailable. Students should discuss the observed bottlenecks, scalability, reliability, and trade-offs of their design.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges in the Library Management System and map the appropriate components to the relevant functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, databases/shards, system-design components, and their interactions.
3. Working Prototype – Implement and integrate the selected components and demonstrate major workflows such as member registration, book search, availability checking, borrowing, returning, and reservation.
4. Performance Evaluation – Report measurable metrics including average response time, throughput, success/error rate, resource usage, and relevant component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.
5. Scalability & Failure Analysis – Demonstrate system behaviour when workload or system size increases and under at least one relevant failure scenario. Evaluate shard distribution, load distribution, and the impact of adding/removing servers or shards where applicable.
6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental results, trade-offs, and limitations, followed by an end-to-end demonstration.

Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)
B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024
III Year CSE (V Semester) Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

4. Scalable Inventory Management System

Design and develop a prototype of a scalable Inventory Management System supporting product registration, inventory tracking, stock availability, warehouse management, purchase and sales orders, stock transfers, and low-stock alerts. The system should be designed to handle increasing numbers of products, warehouses, transactions, and concurrent requests while providing efficient and reliable inventory operations.

The following system-design components are available for incorporation: Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the inventory functionalities is not provided. Students must analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified.

The system should support functionalities such as product search and registration, real-time stock availability, warehouse-wise inventory management, purchase and sales order processing, stock transfers between warehouses, and low-stock notifications. Students should develop an overall system architecture, design and implement the selected components, and integrate them into a working prototype. Inventory and transaction data should be capable of being partitioned across multiple database shards, while consistent hashing may be considered for distributing products, warehouses, or requests across servers. A web crawler may be used with simulated supplier websites to collect product information, supplier prices, or availability.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average response time, throughput (requests/second), success/error rate, and resource usage. Component-specific metrics should also be reported where applicable—for example, request rejection rate for the rate limiter, data distribution across

shards, load distribution across servers for consistent hashing, lookup latency for the key-value store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and accuracy for autocomplete.

Students should evaluate the system under at least three workload levels (for example, low, medium, and high numbers of concurrent requests) and present the measured results using tables or graphs. Where appropriate, students should compare the proposed design with a simple baseline implementation. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of products or transactions, adding a warehouse/server or database shard, or making one server unavailable. Students should evaluate stock consistency, shard distribution, load distribution, and the impact of adding or removing servers/shards, and discuss the observed bottlenecks, scalability, reliability, and design trade-offs.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges in the Inventory Management System and map the appropriate components to the relevant functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, warehouses, databases/shards, system-design components, and their interactions.
3. Working Prototype – Implement and integrate the selected components and demonstrate major workflows such as product registration, stock updates, inventory search, order processing, and stock transfer.
4. Performance Evaluation – Report measurable metrics including average response time, throughput, success/error rate, resource usage, and relevant component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.
5. Scalability & Failure Analysis – Demonstrate system behaviour when workload or system size increases and under at least one relevant failure scenario. Evaluate shard distribution, load distribution, and the impact of adding/removing servers or shards where applicable.

6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental results, trade-offs, and limitations, followed by an end-to-end demonstration.

Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)
B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024
III Year CSE (V Semester) Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

5. Scalable Customer Relationship Management System

Design and develop a prototype of a scalable Customer Relationship Management (CRM) System supporting customer registration and authentication, customer profile management, lead and opportunity management, interaction tracking, sales activities, customer support, and notifications. The system should be designed to handle increasing numbers of customers, leads, interactions, support requests, and concurrent users while providing efficient and reliable access to customer information.

The following system-design components are available for incorporation: Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the CRM functionalities is not provided. Students must analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified.

The system should support functionalities such as customer search and registration, profile management, lead and opportunity tracking, sales representative assignment, interaction and communication history, support-ticket management, and customer notifications. Students should develop an overall system architecture, design and implement the selected components, and integrate them into a working prototype. Customer, lead, and interaction data should be capable of being partitioned across multiple database shards, while consistent hashing may be considered for distributing customers, requests, or services across servers. A web crawler may be used with simulated external sources to collect publicly available company or product information relevant to customer or lead management.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average response time, throughput (requests/second),

success/error rate, and resource usage. Component-specific metrics should also be reported where applicable—for example, request rejection rate for the rate limiter, data distribution across shards, load distribution across servers for consistent hashing, lookup latency for the key-value store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and accuracy for autocomplete.

Students should evaluate the system under at least three workload levels (for example, low, medium, and high numbers of concurrent requests) and present the measured results using tables or graphs. Where appropriate, students should compare the proposed design with a simple baseline implementation. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of customers or interactions, adding a database shard or CRM server, or making one server unavailable. Students should evaluate data distribution, load distribution, search performance, and the impact of adding or removing servers/shards, and discuss the observed bottlenecks, scalability, reliability, and design trade-offs.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges in the CRM System and map the appropriate components to the relevant functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, databases/shards, system-design components, and their interactions.
3. Working Prototype – Implement and integrate the selected components and demonstrate major workflows such as customer registration, customer search, lead creation, opportunity tracking, interaction management, and support-ticket processing.
4. Performance Evaluation – Report measurable metrics including average response time, throughput, success/error rate, resource usage, and relevant component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.
5. Scalability & Failure Analysis – Demonstrate system behaviour when workload or system size increases and under at least one relevant failure scenario. Evaluate data/shard distribution, load distribution, search performance, and the impact of adding/removing servers or shards.

6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental results, trade-offs, and limitations, followed by an end-to-end demonstration.

Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)
B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024
III Year CSE (V Semester) Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

6. Scalable Human Resource Management System

Design and develop a prototype of a scalable Human Resource Management (HRM) System supporting employee registration and authentication, employee profile management, attendance tracking, leave management, payroll processing, recruitment, and performance management. The system should be designed to handle increasing numbers of employees, attendance records, leave requests, payroll transactions, and concurrent users while providing efficient and reliable access to HR services.

The following system-design components are available for incorporation: Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the HR functionalities is not provided. Students must analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified.

The system should support functionalities such as employee registration and search, employee profile management, attendance and leave tracking, payroll processing, recruitment and candidate management, performance tracking, and HR notifications. Students should develop an overall system architecture, design and implement the selected components, and integrate them into a working prototype. Employee, attendance, payroll, and recruitment data should be capable of being partitioned across multiple database shards, while consistent hashing may be considered for distributing employees, requests, or services across servers. A web crawler may be used with simulated external job portals to collect job listings or publicly available recruitment information.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average response time, throughput (requests/second),

success/error rate, and resource usage. Component-specific metrics should also be reported where applicable—for example, request rejection rate for the rate limiter, data distribution across shards, load distribution across servers for consistent hashing, lookup latency for the key-value store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and accuracy for autocomplete.

Students should evaluate the system under at least three workload levels (for example, low, medium, and high numbers of concurrent requests) and present the measured results using tables or graphs. Where appropriate, students should compare the proposed design with a simple baseline implementation. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of employees or attendance records, adding a database shard or HR server, or making one server unavailable. Students should evaluate data/shard distribution, load distribution, search performance, and the impact of adding or removing servers/shards, and discuss the observed bottlenecks, scalability, reliability, and design trade-offs.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges in the HRM System and map the appropriate components to the relevant functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, databases/shards, system-design components, and their interactions.
3. Working Prototype – Implement and integrate the selected components and demonstrate major workflows such as employee registration, employee search, attendance/leave management, payroll processing, and recruitment.
4. Performance Evaluation – Report measurable metrics including average response time, throughput, success/error rate, resource usage, and relevant component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.
5. Scalability & Failure Analysis – Demonstrate system behaviour when workload or system size increases and under at least one relevant failure scenario. Evaluate data/shard distribution, load distribution, search performance, and the impact of adding/removing servers or shards.

6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental results, trade-offs, and limitations, followed by an end-to-end demonstration.

Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)
B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024
III Year CSE (V Semester) Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

7. Scalable Online Banking System

Design and develop a prototype of a scalable Online Banking System supporting customer registration and authentication, account management, balance enquiry, fund transfers, bill payments, transaction history, and notifications. The system should be designed to handle increasing numbers of customers, accounts, transactions, and concurrent requests while maintaining efficient, reliable, and consistent banking operations.

The following system-design components are available for incorporation: Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the banking functionalities is not provided. Students must analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified. The prototype should use synthetic banking data only and should not process real financial information.

The system should support functionalities such as customer registration and authentication, account and balance management, beneficiary management, fund transfers, bill payments, transaction history, and transaction notifications. Students should develop an overall system architecture, design and implement the selected components, and integrate them into a working prototype. Customer, account, and transaction data should be capable of being partitioned across multiple database shards, while consistent hashing may be considered for distributing requests or services across servers. A web crawler may be used with simulated external sources, such as public biller information or exchange-rate data, rather than accessing real banking systems.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average response time, throughput (transactions/requests per second), success/error rate, and resource usage. Component-specific metrics should also be reported where applicable—for example, request rejection rate for the rate limiter, data distribution across shards, load distribution across servers for consistent hashing, lookup latency for the key-value store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and accuracy for autocomplete.

Students should evaluate the system under at least three workload levels (for example, low, medium, and high numbers of concurrent requests) and present the measured results using tables or graphs. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of customers or transactions, adding a database shard or banking server, or making one server unavailable. Students should evaluate shard distribution, load distribution, transaction consistency, and the impact of adding/removing servers or shards, and discuss observed bottlenecks, scalability, reliability, and design trade-offs.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges in the Online Banking System and map the appropriate components to the relevant functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, databases/shards, system-design components, and their interactions.
3. Working Prototype – Implement and integrate the selected components and demonstrate major workflows such as customer authentication, account enquiry, fund transfer, bill payment, and transaction history.
4. Performance Evaluation – Report measurable metrics including average response time, throughput, success/error rate, resource usage, and relevant component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.
5. Scalability & Failure Analysis – Demonstrate system behaviour when workload or system size increases and under at least one relevant failure scenario. Evaluate transaction consistency, shard distribution, load distribution, and the impact of adding/removing servers or shards.

6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental results, trade-offs, limitations, and security considerations, followed by an end-to-end demonstration using synthetic data.

Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)
B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024
III Year CSE (V Semester) Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

8. Scalable Student Information System

Design and develop a prototype of a scalable Student Information System (SIS) supporting student registration and authentication, student profile management, course registration, attendance tracking, examination and grade management, timetable access, and academic notifications. The system should be designed to handle increasing numbers of students, courses, academic records, and concurrent requests while providing efficient, reliable, and consistent access to academic services.

The following system-design components are available for incorporation: Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the SIS functionalities is not provided. Students must analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified.

The system should support functionalities such as student registration and authentication, student and course search, course registration, attendance management, examination and grade management, timetable access, and academic notifications. Students should develop an overall system architecture, design and implement the selected components, and integrate them into a working prototype. Student, course, attendance, and academic records should be capable of being partitioned across multiple database shards, while consistent hashing may be considered for distributing requests or services across servers. A web crawler may be used with simulated external sources, such as university information portals, course resources, or scholarship and internship listings.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average response time, throughput (requests/second),

success/error rate, and resource usage. Component-specific metrics should also be reported where applicable—for example, request rejection rate for the rate limiter, data distribution across shards, load distribution across servers for consistent hashing, lookup latency for the key-value store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and accuracy for autocomplete.

Students should evaluate the system under at least three workload levels (for example, low, medium, and high numbers of concurrent requests) and present the measured results using tables or graphs. Where appropriate, students should compare the proposed design with a simple baseline implementation. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of students or course registrations, adding a database shard or application server, or making one server unavailable. Students should evaluate data/shard distribution, load distribution, search performance, and the impact of adding or removing servers/shards, and discuss observed bottlenecks, scalability, reliability, and design trade-offs.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges in the SIS and map the appropriate components to the relevant functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, databases/shards, system-design components, and their interactions.
3. Working Prototype – Implement and integrate the selected components and demonstrate major workflows such as student registration, course search, course registration, attendance/grade management, and timetable access.
4. Performance Evaluation – Report measurable metrics including average response time, throughput, success/error rate, resource usage, and relevant component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.
5. Scalability & Failure Analysis – Demonstrate system behaviour when workload or system size increases and under at least one relevant failure scenario. Evaluate data/shard distribution, load distribution, search performance, and the impact of adding/removing servers or shards.

6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental results, trade-offs, and limitations, followed by an end-to-end demonstration.

Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam – 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)
B.E DEGREE EXAMINATIONS, NOV/DEC 2026 - Regulations - 2024
III Year CSE (V Semester) Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Mini Project

9. Scalable Hotel Management System

Design and develop a prototype of a scalable Hotel Management System supporting guest registration and authentication, room search and availability, room booking, check-in and check-out, housekeeping management, billing, and guest notifications. The system should be designed to handle increasing numbers of guests, rooms, bookings, and concurrent requests while providing efficient, reliable, and consistent hotel operations.

The following system-design components are available for incorporation: Rate Limiter, Sharding, Consistent Hashing, Key-Value Store, Unique ID Generator, URL Shortener, Web Crawler, and Autocomplete. The mapping between these components and the hotel functionalities is not provided. Students must analyse the requirements and determine which component(s) are appropriate for each functionality. A component may be used for multiple functionalities, and a functionality may require multiple components, provided the choices are properly justified.

The system should support functionalities such as guest registration and authentication, hotel and room search, room availability checking, reservations, check-in/check-out, housekeeping status management, billing, and guest notifications. Students should develop an overall system architecture, design and implement the selected components, and integrate them into a working prototype. Guest, room, and booking data should be capable of being partitioned across multiple database shards, while consistent hashing may be considered for distributing hotels, rooms, or requests across servers. A web crawler may be used with simulated external hotel websites to collect information such as room rates, availability, amenities, or reviews.

The project should include quantitative performance evaluation. Students should measure appropriate metrics such as average response time, throughput (requests/second), success/error rate, and resource usage. Component-specific metrics should also be reported where applicable—for example, request rejection rate for the rate limiter, data distribution across shards, load distribution across servers for consistent hashing, lookup latency for the key-value

store, ID generation rate and collision count for the unique ID generator, redirection latency for the URL shortener, crawling throughput for the web crawler, and suggestion response time and accuracy for autocomplete.

Students should evaluate the system under at least three workload levels (for example, low, medium, and high numbers of concurrent requests) and present the measured results using tables or graphs. Where appropriate, students should compare the proposed design with a simple baseline implementation. The evaluation should also include at least one scalability or failure scenario, such as increasing the number of guests or bookings, adding a database shard or hotel server, or making one server unavailable. Students should evaluate room/booking consistency, shard distribution, load distribution, and the impact of adding or removing servers/shards, and discuss observed bottlenecks, scalability, reliability, and design trade-offs.

Project Deliverables

1. Requirement Analysis & Component Mapping – Identify the system-design challenges in the Hotel Management System and map the appropriate components to the relevant functionalities, with justification.
2. System Architecture & Design – Provide an architecture diagram showing the major services, databases/shards, system-design components, and their interactions.
3. Working Prototype – Implement and integrate the selected components and demonstrate major workflows such as guest registration, room search, availability checking, booking, check-in/check-out, and billing.
4. Performance Evaluation – Report measurable metrics including average response time, throughput, success/error rate, resource usage, and relevant component-specific metrics under at least three workload levels. Include tables/graphs and, where appropriate, comparison with a baseline.
5. Scalability & Failure Analysis – Demonstrate system behaviour when workload or system size increases and under at least one relevant failure scenario. Evaluate room/booking consistency, data/shard distribution, load distribution, and the impact of adding/removing servers or shards.
6. Project Report & Demonstration – Submit a report documenting requirements, component selection, architecture, algorithms, implementation, experimental results, trade-offs, and limitations, followed by an end-to-end demonstration.